

Understanding Multi-Query Attacks on Key-Then-Hash Functions

Jonathan Fuchs

Radboud University, Nijmegen, The Netherlands
jonathan.fuchs@ru.nl

Abstract. We present multi-query attacks on key-then-hash (KTH) functions in the blinded keyed hash model that achieve an advantage growing quadratically in the number of queries up to a small constant factor from the information-theoretic upper bound. We introduce three families of attacks. Catch attacks exploit the group structure of the digest space and achieve deterministic success with $2\sqrt{\varepsilon^{-1}}$ queries. Group attacks embed high-probability differentials into subgroups of the message space of quadratic advantage. Translation attacks exploit offset-invariance to linearly scale any existing attack. Our attacks apply in two concrete settings: with Δ fixed to 0, they target the compression phase of farfalle-based primitives such as Xooff, and with Δ as a free parameter, they target deck-based wide block cipher constructions such as the double-decker. We connect optimal query set construction to results in additive combinatorics and generalize our results to concatenated KTH functions. Experiments on NH and XOODOO[3] show our attacks reach an advantage within a factor 2^4 of the theoretical bound. Our analysis reveals that for bit-sliced permutations with degree-2 round functions, solution set overlap is inherent, limiting but not preventing the attacker from approaching the bound. Our experiments highlight that trail cores with a large number of active columns in the last round are particularly dangerous for KTH functions, introducing a new criterion for the design of permutations used in such constructions.

Keywords: message authentication codes · key-then-hash · multi-query security · offset-invariance · protected hash · blinded keyed hash

1 Introduction

Universal hash functions (UHF) are a family F of variable-input-length functions that are selected by a key. They satisfy the universality property defined by Stinson in [Sti95]. This property upper bounds the probability, taken over all keys, that two different messages result in two digests that differ exactly by the value Δ , i.e.,

$$\max_{m \neq m'} \Pr_{k \xleftarrow{\$} \mathcal{K}} [F_k(m) - F_k(m') = \Delta] \leq \varepsilon.$$

When this property holds for any value of Δ we say that F is ε - Δ -universal and when Δ is fixed to the value 0 we say F is ε -universal. Protected hash is a generic construction first introduced by Bellare, Canetti and Krawczyk in [BCK96]. It turns a fixed-input-length pseudorandom function (FIL-PRF) P into a variable-input-length pseudorandom function (VIL-PRF) P' by first feeding the input into a UHF and then taking the digest as the input of the FIL-PRF:

$$P'_{k_1, k_2}(m) = P_{k_1}(F_{k_2}(m)).$$

The security of this construction depends on the universality of the keyed hash function and the distinguishing advantage of an attacker to distinguish the FIL-PRF from a random oracle. The blinded keyed hash (bkh) security model, introduced by Daemen et al. in [DHP⁺18] and then formalized by Gunesing et al. in [GDM19], is a model whose security is defined as distinguishing advantage between the following function and a random oracle.

$$P'_{k_1, k_2}(m) = P_{k_1}(F_{k_2}(m) + \Delta).$$

Similarly, the security of this construction depends on the Δ -universality of F and the distinguishing advantage of the attacker on P . When the output space of P is large enough, the most efficient way for an attacker to distinguish P' from a random oracle is to generate collisions at the output of F (or a difference Δ). Achieving this, for an ε - Δ -universal/ ε -universal F , will require from the attacker at least $\sqrt{\varepsilon^{-1}}$ queries.

Key-then-hash (KTH) functions are a family of UHF formalized by Fuchs et al. in [FRD23]. They are the family of keyed hash functions that add the secret key to the input string using some group addition and then process the resulting string with an unkeyed (public) function. In their paper, they show two constructions, a serial and a parallel one, that build a KTH function using a public permutation. They prove that the universality of such constructions is upper bounded by the maximum differential probability of the underlying public permutation. Later on, Ghosh et al. generalized this result for the parallel construction to include public functions, i.e., functions that are not necessarily a bijection [GFAD23].

In the bkh security model, the advantage of an attacker is based on its ability to choose a set of queries Q that maximize the probability of generating a collision at the input of the FIL-PRF. For a query set of size $\#Q$, applying the union bound over all pairs gives an upper bound on the attacker's advantage of $\binom{\#Q}{2}\varepsilon$ which grows quadratically in the size of Q . However, the tightness of this quadratic growth from the attacker's perspective has not been investigated.

In this paper, we investigate under what conditions an attacker can approach this quadratic growth and how close practical attacks get on concrete instantiations. We present a series of attacks that make use of the underlying group structure of KTH functions in order to achieve an advantage that grows quadratically in the number of queries. We apply our attacks on the parallel construction instantiated with integer multiplication modulo 2^{2w} , which is equivalent to NH in UMAC [BHK⁺99], and on Xoodoo[3], the public permutation used in primitives such as Xooff [DHVV18b] and Xoodyak [DHP⁺20]. Our experiments show that we can achieve an advantage that grows almost quadratically in the number of queries, only differing from the theoretical bound by small factors.

Our attacks are relevant to two families of concrete constructions. When the offset Δ is fixed to 0, collisions at the output of the UHF directly compromise the compression phase of farfalle-based primitives such as Xooff [DHVV18b]. When Δ is a free parameter chosen by the attacker, the attacks apply to deck-based wide block cipher constructions such as the double-decker and docked-double-decker [GDM19]. The design criteria we derive are therefore relevant to both settings.

1.1 Our Contributions

Our contributions can be summarized as the following:

- We introduce a type of attack that we call “catch attacks”. These attacks exploit the fact that we can add an offset to the digest and succeed in generating a collision in a fixed number of queries. The required number of queries depends on the type of the group the digest space forms. When all conditions are met, this type of attack requires only $2\sqrt{\varepsilon^{-1}}$ queries for an ε - Δ -universal KTH function.

- We link the problem of finding query sets over equal-length messages that achieve optimal advantage to a well-known result in additive combinatorics. We are able to show that the size of an optimal set is upper bounded by the size of the set of differentials that achieve maximal differential probability.
- We introduce a type of attack that we call “group attacks”. These attacks exploit the fact that the inputs to KTH functions are strings of group elements. We can embed a set of differentials with probability approximately ε into a subgroup of the message space and potentially gain a quadratic advantage in the number of queries. We also generalize this attack by applying the same “catching” mechanism of catch attacks, which allows the attacker to gain a deterministic quadratic increase, depending only on the properties of the digest space.
- We show that the translation invariance of KTH functions may allow an attacker to linearly scale their attack by repeating translated copies of their original query set. We characterize the conditions required for such a linear scaling to succeed and provide a greedy algorithm that tries to achieve it.
- We generalize the universality of concatenating multiple independent calls of a UHF to multi-query attacks in the bkh model with KTH setting. We show that the advantage that an attacker has against a n times concatenated KTH function is upper bounded by the product of the advantage against a single iteration of the KTH function and ε^{n-1} .
- We introduce two types of generic attacks against the parallel KTH construction. One attack makes use of a query set that contains only strings of different length. The other attack makes use of a query set that contains only long, equal-length messages. Both of these attacks have a potential quadratic increase in the number of queries. We also prove that these attacks have a lower bound of success probability that scales linearly with the number of queries.
- We implement these attacks on NH[8], NH[16] and XODOO[3] and show that they succeed in varying degrees. Some of the attacks reach a success probability that is only a small factor away from the theoretical quadratic upper bound.

Table 1: Summary of multi-query attacks. ε denotes the universality parameter. The gap is the factor between the attack’s success probability and the upper bound $\binom{\#Q}{2}\varepsilon$.

	Catch (Thm. 1)	Group 2.0 (Thm. 4)
Applies to	Any KTH	Any KTH
$\#Q$	$2\sqrt{\varepsilon^{-1}}$	$\# \langle A_{b,\ell} \rangle (\#H_1 + \#H_2)$
Pr[success]	1	$\approx \varepsilon \#A_{b,\ell} \# \langle A_{b,\ell} \rangle \# \langle B \rangle$
Gap from bound	$\geq \sqrt{2}$	≥ 2
	Unequal-length (Prop. 3)	Two-block (Prop. 4)
Applies to	Parallel[f] only	Parallel[f] only
$\#Q$	ℓ	$\ell + 1$
Pr[success]	$\geq 1 - (1 - \text{MIP}_f)^{\ell-1}$	$\geq 1 - (1 - \text{MDP}_f)^{\ell-1}$
Gap from bound	Linear growth only	Linear growth only

2 Preliminaries

In this section we define the basic concepts and notation that we will use throughout the paper.

2.1 Notation

The keyed hash functions we study in this paper take strings of variable length as input. The strings are composed of elements of the abelian group $(G, +)$ with 0 denoting the neutral element. We refer to single elements of G as blocks. The space of all strings composed of ℓ blocks is denoted as G^ℓ . Strings are denoted in bold, i.e., $\mathbf{m} \in G^\ell$, with blocks m_i indexed from 1 to ℓ . κ denotes the maximum acceptable length for input strings. The space of keys is defined as G^κ . The space containing all strings of length 1 to κ is denoted as $\text{BS}(G, \kappa) = \cup_{\ell=1}^{\kappa} G^\ell$. $|\mathbf{m}| \in \mathbb{Z}_{\geq 0}$ denotes the number of blocks in a given string $\mathbf{m}$. Finally, a keyed hash function is denoted as $F: G^\kappa \times \text{BS}(G, \kappa) \rightarrow G'$, with G' another abelian group $(G', +)$ that we refer to as the digest space. We will discuss sets of input and output differences that we denote by A or B respectively. We use subscripts to specify restrictions on the differences. We denote by A_b the set of input differences that are *compatible* (see Section 2.5) with an output difference b . We denote by $A_{b,\ell}$ the set of input differences that contains only differences between messages of length ℓ that are compatible with b . We denote by $\langle A \rangle \leq G$ the subgroup of G generated by A . We denote by $A - A$ the set containing the pairwise differences of the elements of A .

2.2 Universal Hash Functions

We speak of the universality or Δ -universality of a keyed hash function. We provide Stinson's definitions of these properties, but adapted to our notation.

Definition 1 (ε -universality [Sti95]). A keyed hash function F is said to be ε -universal if for any distinct strings $\mathbf{m}, \mathbf{m}^* \in \text{BS}(G, \kappa)$

$$\Pr[F_{\mathbf{k}}(\mathbf{m}) = F_{\mathbf{k}}(\mathbf{m}^*)] \leq \varepsilon,$$

with the probability taken over the key space G^κ .

Definition 2 (ε - Δ -universality [Sti95]). A keyed hash function F is said to be ε - Δ -universal if for any distinct strings $\mathbf{m}, \mathbf{m}^* \in \text{BS}(G, \kappa)$ and for all $\Delta \in G'$

$$\Pr[F_{\mathbf{k}}(\mathbf{m}) - F_{\mathbf{k}}(\mathbf{m}^*) = \Delta] \leq \varepsilon,$$

with the probability taken over the key space G^κ .

2.3 Blinded Keyed Hash Model

The blinded keyed hash security (bkh) of a keyed hash function $F_{\mathbf{k}}$ is defined as the advantage of an adversary distinguishing between two worlds:

- Real world: $\mathcal{RO}_1(F_{\mathbf{k}}(m) + \Delta)$ with $\mathbf{k} \xleftarrow{\$} \mathcal{K}$ and a secret random oracle $\mathcal{RO}_1$;
- Ideal world: $\mathcal{RO}_2(m, \Delta)$ with a secret random oracle $\mathcal{RO}_2$.

It was first published in [DHP⁺18] then formalized in [GDM19]. We depict the setup in Figure 1. We see random oracles in both the real and ideal world. In the real world, the adversary does not obtain the output of $F_{\mathbf{k}}$ directly, but rather the output of a random oracle applied to this digest, offset by a value of her choice $\Delta \in G'$. In the ideal world, the

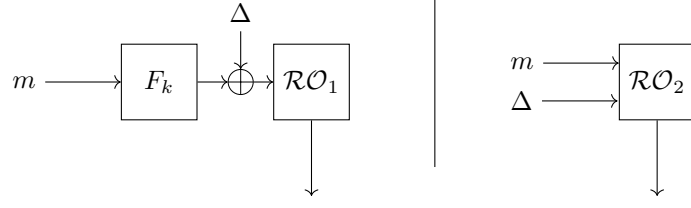


Figure 1: The distinguishing setup for bkh security, with the real world on the left and ideal world on the right. We adopted this figure from [GDM19].

adversary obtains the output of a random oracle applied to an injective encoding of the message and offset.

In the real world, queries that lead to different inputs to $\mathcal{RO}_1$ will have completely random responses. However, the responses of pairs of queries that collide at the input of $\mathcal{RO}_1$ will be equal. Hence, the occurrence of these collisions is the only thing that the adversary may learn about F from queries to the real world. From [GDM19] we know that for a query set $Q \subset \text{BS}(\mathcal{G}, \kappa) \times \mathcal{G}'$, the bkh advantage, denoted as $\text{Adv}_F^{\text{bkh}}(Q)$, is equal to:

$$\text{Adv}_F^{\text{bkh}}(Q) = \Pr[\text{Bad}_F^{\text{bkh}}(Q)],$$

where $\text{Bad}_F^{\text{bkh}}(Q)$ is the following predicate:

$$\exists((\mathbf{m}, \Delta), (\mathbf{m}', \Delta')) \in Q^2: F_{\mathbf{k}}(\mathbf{m}) + \Delta = F_{\mathbf{k}}(\mathbf{m}') + \Delta' \wedge (\mathbf{m}, \Delta) \neq (\mathbf{m}', \Delta').$$

The probability $\Pr[\text{Bad}_F^{\text{bkh}}(Q)]$ is taken over the key space $\mathcal{K}$ and is given by the following ratio:

$$\frac{\#\{\mathbf{k} \in \mathcal{K} \mid \text{Bad}_F^{\text{bkh}}(Q)\}}{\#\mathcal{K}}.$$

2.4 Key-Then-Hash Functions

KTH functions are UHF of the form $F_{\mathbf{k}}(\cdot) = F(\mathbf{k} + \cdot)$ introduced in [FRD23]. Given two strings $\mathbf{m}, \mathbf{m}^* \in \text{BS}(\mathcal{G}, \kappa)$ with $|\mathbf{m}| \leq |\mathbf{m}^*|$, we define the sum of the two strings as:

$$\mathbf{m} + \mathbf{m}^* = (m_1 + m_1^*, m_2 + m_2^*, \dots, m_{|\mathbf{m}|} + m_{|\mathbf{m}|}^*).$$

Hence, the resulting string is of length $|\mathbf{m}|$.

F is considered a public function and therefore the secret key is only used once when it is added to the input string. There are currently two proposed constructions to build KTH functions.

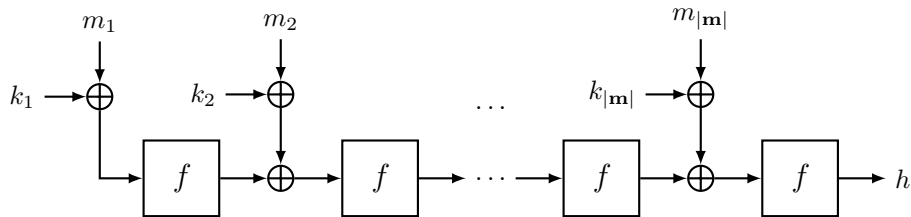


Figure 2: The serialization $\text{Serial}[f]$ [FRD23]

The first one processes the input blocks in a serial way, depicted in Figure 2. It builds a UHF in a Pelican-MAC-like [DR05] structure. It is proven that when the underlying function f is a bijection, the universality and Δ -universality of $\text{Serial}[f]$ is MDP_f [FRD23, Theorem 1] (See Section 2.5).

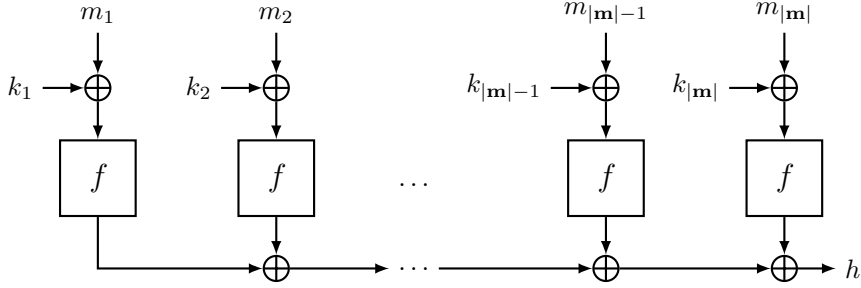


Figure 3: The parallelization $\text{Parallel}[f]$ [FRD23]

The second construction processes the input blocks in parallel. It is an idealization of the compression phase of Farfalle [BDH⁺17]. We depict it in Figure 3. It is proven that when f is a bijection, $\text{Parallel}[f]$ is MNDP_f -universal and MDP_f - Δ -universal [FRD23, Theorems 2&3]. When f is not a bijection, $\text{Parallel}[f]$ is $\max\{\text{MDP}_f, \text{MIP}_f\}$ - Δ -universal [GFAD23, Theorem 1] (See Section 2.5).

2.5 Differential Probability and Image Probability

The universality of the UHFs we study depends on the differential probability of the underlying fixed-length public functions that they are built upon. We therefore define differential probability and some other important quantities.

Definition 3 (Differential probability). The differential probability of a differential (a, b) over a function $f: G \rightarrow G'$, denoted as $\text{DP}_f(a, b)$, is:

$$\text{DP}_f(a, b) = \frac{\#\{x \in G \mid f(x + a) - f(x) = b\}}{\#G}.$$

We say that a propagates to b with probability $\text{DP}_f(a, b)$.

If $\text{DP}_f(a, b) > 0$, we call input difference a and output difference b *compatible* through f . In our bounds $\max_{a \neq 0, b} \text{DP}_f(a, b)$ plays an important role, and we denote it by MDP_f . A useful quantity is the square of the Euclidean norm of DP for fixed input difference a , given by $\sum_b \text{DP}_f^2(a, b)$. We denote the maximum of this quantity over all input differences by MNDP_f , hence $\text{MNDP}_f = \max_{a \neq 0} \sum_b \text{DP}_f^2(a, b)$.

Another useful quantity is the image probability of a (public) function f .

Definition 4 (Image probability). Let $f: G \rightarrow G'$ be a public function. The image probability of an output $\Delta \in G'$ of f , denoted as $\text{IP}(\Delta)$, is the number of inputs that f maps to Δ divided by the total number of possible inputs, namely,

$$\text{IP}(\Delta) = \frac{\#\{x \in G \mid f(x) = \Delta\}}{\#G}.$$

Similarly, we denote the maximum image probability over all outputs as $\text{MIP}_f = \max_{\Delta} \text{IP}(\Delta)$ and $\text{MNIP}_f = \max_b \sum_b \text{IP}(b)^2$.

Lastly, Fuchs et al. generalize the notion of differences and differentials to variable-input-length functions. We quote these definitions verbatim.

Definition 5 (Difference between two strings [FRD23]). We define the difference between two strings $\mathbf{m}, \mathbf{m}^*$ with $|\mathbf{m}| \leq |\mathbf{m}^*|$ as the pair $(\mathbf{a}, \lambda) \in \mathbb{G}^{|\mathbf{m}|} \times \mathbb{Z}_{\geq 0}$ where $\mathbf{a} = (m_1 - m_1^*, m_2 - m_2^*, \dots, m_{|\mathbf{m}|} - m_{|\mathbf{m}|}^*)$ and $\lambda = |\mathbf{m}^*| - |\mathbf{m}|$.

When $\lambda = 0$ we say a difference is *equal-length* and otherwise we say it is *unequal-length*. Now, we define the probability of differentials over a KTH function F .

Definition 6 (Generalized differentials and their DP [FRD23]). Given an input difference $(\mathbf{a}, \lambda)$ and an output difference Δ , the differential probability of the differential $(\mathbf{a}, \lambda, \Delta)$ over F , denoted as $\text{DP}_F(\mathbf{a}, \lambda, \Delta)$ is given by:

$$\text{DP}_F(\mathbf{a}, \lambda, \Delta) = \frac{\#\{\mathbf{k} \in \mathbb{G}^\kappa \mid F(\mathbf{a} + \mathbf{k}) - F(0^{|\mathbf{a}|+\lambda} + \mathbf{k}) = \Delta\}}{\#\mathbb{G}^\kappa}.$$

In [FRD23], these generalized differentials were used to prove upper bounds on the universality of KTH constructions. In this paper, we use them from the attacker’s perspective: we construct query sets whose pairwise differences correspond to differentials with probability ε .

2.6 Differential Propagation In Iterated Permutations

An iterated permutation is a permutation constructed through the repeated application of a round function: Let $\rho: \mathbb{G} \rightarrow \mathbb{G}$ be a bijection over $\mathbb{G}$, the n -round iterated permutation $f: \mathbb{G} \rightarrow \mathbb{G}$ is given by applying ρ to itself n times, i.e., $f(x) = (\rho \circ \rho \circ \dots \circ \rho)(x)$. Differentials over iterated permutations are often studied by means of their differential trails.

Definition 7 (Differential trail). An r -round differential trail, denoted as T , over the iterated permutation $f_k(m) = (\rho \circ \rho \circ \dots \circ \rho)(m + k)$, is a sequence of $r + 1$ differences: an input difference, $r - 1$ intermediate differences and an output difference, where the round differentials (q_i, q_{i+1}) have non-zero DP_ρ , namely,

$$T = (q_1, q_2, q_3, \dots, q_{r-1}, q_r, q_{r+1}) \text{ with } \text{DP}_\rho(q_i, q_{i+1}) > 0 \text{ for } 1 \leq i < r.$$

The differential probability of a trail, denoted $\text{DP}_f(T)$, is the probability that a random pair with input difference q_1 propagates via intermediate differences $q_2, q_3, \dots$ to output difference q_{r+1} . Furthermore, the probability of differentials over f is given by that of differential trails: $\text{DP}_f(a, b) = \sum_{T \in \mathcal{T}} \text{DP}_f(T)$, where $\mathcal{T}$ is the set of all differential trails starting with a and ending with b .

A useful concept when studying differential propagation is the *restriction weight*.

Definition 8 (Restriction weight of a differential [Dae95]). The restriction weight of a differential $\text{DP}(a, b) > 0$ is defined as $w(a, b) = -\log_2 \text{DP}(a, b)$.

Definition 9 (Restriction weight of a differential trail [Dae95]). The restriction weight of a differential trail $T = (q_0, q_1, \dots, q_{r-1})$ is defined as $w(T) = \sum_i w(q_{i-1}, q_i)$, hence the sum of the restriction weights of its round differentials.

Papers on trail search often report on large sets of trails with common features rather than individual trails. These sets are called *trail cores*.

Definition 10 (Differential trail core [DV12]). An r -round differential trail core, denoted as $\tilde{T}$, is a set of differential trails over r rounds with a shared core of intermediate differences $(q_2, \dots, q_r)$ with $\text{DP}_\rho(q_i, q_{i+1}) > 0$ for $1 \leq i \leq r$.

The trail cores of XOODOO[3] catalogued in [DHVV18a] will serve as the building blocks for our group attacks in Section 6. The number of active columns in a trail core determines the size of the compatible input difference set, which directly controls the efficiency of our attacks.

2.7 Solution Sets

In our proofs, we sometimes use the subset of the key space for which a digest collision happens for a fixed pair of queries:

Definition 11 (Solution Set). Let $F: \text{BS}(\mathbb{G}, \kappa) \times \mathbb{G}^\kappa \rightarrow \mathbb{G}'$ be a key-then-hash function. Let $\mathbf{m}, \mathbf{m}^* \in \text{BS}(\mathbb{G}, \kappa)$ be two input strings with $|\mathbf{m}| \leq |\mathbf{m}^*|$. Let $\Delta \in \mathbb{G}'$ be an output difference. The solution set of $\mathbf{m}, \mathbf{m}^*$ and Δ , denoted as $\mathcal{S}(\mathbf{m}, \mathbf{m}^*, \Delta)$, is the subset of the key space for which the difference in the digests of $\mathbf{m}$ and $\mathbf{m}^*$ is equal to Δ , i.e.,

$$\{\mathbf{k} \in \mathbb{G}^\kappa \mid F(\mathbf{m} + \mathbf{k}) - F(\mathbf{m}^* + \mathbf{k}) = \Delta\}.$$

We may also use a shortcut in notation when talking about the solution set of a set of queries, denoted as $\mathcal{S}(Q)$, which is equal to the union of all solution sets possible in the query set:

$$\begin{aligned} \mathcal{S}(Q) &= \bigcup_{p \neq q \in Q} \mathcal{S}(\{p, q\}) \\ &= \bigcup_{(\mathbf{m}, \Delta) \neq (\mathbf{m}^*, \Delta^*) \in Q} \mathcal{S}(\mathbf{m}, \mathbf{m}^*, \Delta^* - \Delta). \end{aligned}$$

3 Generic Attacks on KTH Functions

We present, in this section, a number of attacks that work for any KTH function in the bkh setting. Their efficiency and success probability depend on the kind of group structure over the set of differentials that achieve maximum differential probability.

3.1 Catch Attacks

We exploit the group structure of the set of compatible output differences to a fixed input difference to design a generic attack with deterministic success. Its cost depends on the specific properties of that set.

Theorem 1 (Catch Attacks). *Let F be a KTH function. Let $(\mathbf{a}, \lambda) \in \text{BS}(\mathbb{G}, \kappa) \times \mathbb{Z}_{\geq 0}$ be an input difference. Let $B = \{b \in \mathbb{G}' \mid \text{DP}_F(\mathbf{a}, \lambda, b) > 0\}$ be the set of output differences compatible with $(\mathbf{a}, \lambda)$ through F . Let $H_1, H_2 \leq \mathbb{G}'$ be two subgroups of $\mathbb{G}'$ such that $H_1 + H_2 = \langle B \rangle$. Define the set $Q = \{0^{|\mathbf{a}|+\lambda}\} \times H_1 \cup \{\mathbf{a}\} \times H_2$ as the set of queries to the bkh generating oracle. The following holds:*

$$\Pr[\text{Bad}_F^{\text{bkh}}(Q)] = 1.$$

Proof. There are two types of pairs in Q : those that have an input difference 0 and therefore must propagate to an output difference 0, or those that have an input difference $(\mathbf{a}, \lambda)$ and propagate to an output difference $b \in \langle B \rangle$. Since all pairs with input difference $(\mathbf{a}, \lambda)$ share the same message values, they cannot propagate to a different output difference. Therefore, the events that a pair propagates to an output difference $b \in \langle B \rangle$ and $b^* \in \langle B \rangle$, with $b \neq b^*$, are mutually exclusive. Thus, we get the following derivation:

$$\begin{aligned} \Pr[\text{Bad}_F^{\text{bkh}}(Q)] &= \sum_{q, p \in Q} \Pr[\text{Bad}_F^{\text{bkh}}(\{q, p\})] \\ &= \sum_{b \in \langle B \rangle} \text{DP}_F(\mathbf{a}, \lambda, b) \\ &= \sum_{b \in B} \text{DP}_F(\mathbf{a}, \lambda, b) = 1. \end{aligned}$$

□

Informally, catch attacks work by covering all possible output differences using two subgroups whose sum generates the full compatible output space. Since every key must produce some output difference in this space, every key is caught by at least one pair in the query set, yielding deterministic success. The cost of this deterministic guarantee is a factor $\sqrt{2}$ above the information-theoretic minimum.

Proposition 1. *The lower bound on the size of the query set in catch attacks on an ε - Δ -universal KTH function is*

$$\#Q \geq 2\sqrt{\varepsilon^{-1}}.$$

Proof. We start by proving $\#Q \geq 2\sqrt{\#\langle B \rangle}$. We have that $\#Q = \#H_1 + \#H_2$. The following holds.

$$\#\langle B \rangle = \#(H_1 + H_2) = \frac{\#H_1\#H_2}{\#(H_1 \cap H_2)}.$$

Hence, we get

$$\begin{aligned} \#H_1\#H_2 &= \#\langle B \rangle \#(H_1 \cap H_2) \\ &\geq \#\langle B \rangle. \end{aligned}$$

Then, we apply the arithmetic mean-geometric mean inequality to $\#H_1 + \#H_2$:

$$\begin{aligned} \#H_1 + \#H_2 &\geq 2\sqrt{\#H_1\#H_2} \\ &\geq 2\sqrt{\#\langle B \rangle}. \end{aligned}$$

We prove next that $\#\langle B \rangle \geq \varepsilon^{-1}$. The following two properties hold:

$$\sum_{b \in B} \text{DP}_F(\mathbf{a}, \lambda, b) = 1. \quad (1)$$

$$\forall b \in B \text{ DP}_F(\mathbf{a}, \lambda, b) \leq \varepsilon. \quad (2)$$

The number of terms in the sum of Equation (1) is minimized when all output differences have the maximum possible differential probability. In that case, the number of elements in B is ε^{-1} . Assuming $B = \langle B \rangle$ yields the lower bound. $\square$

Corollary 1. $H_1 \cap H_2 = \{0\}$ and $\#H_1 = \#H_2$ if and only if $\#Q = 2\sqrt{\#\langle B \rangle}$.

In other words, a necessary and sufficient condition for the attack to be optimal is that $B \cong H_1 \oplus H_2$ and $\#H_1 = \#H_2 = \varepsilon^{-\frac{1}{2}}$.

Corollary 2. *Optimal catch attacks use approximately a factor $\sqrt{2}$ more queries than the theoretical minimum.*

Corollary 3. *When G' allows such decomposition (e.g., $G' = \{0, 1\}^{2n}$ with $n \in \mathbb{Z}_{\geq 1}$), choosing $B = G'$ yields a deterministic birthday bound.*

3.2 Maximum Size of Optimal Query Set for Equal-Length Messages

We can use standard additive combinatorics arguments to prove an upper bound on the size of query sets that achieve optimal advantage through the usage of equal-length messages. Assuming $\Pr[\text{Bad}_F^{\text{bkh}}(Q)] = \binom{\#Q}{2}\varepsilon$ the following must hold: $\forall p \neq q \in Q \Pr[\text{Bad}_F^{\text{bkh}}(\{p, q\})] = \varepsilon$. In other words, each of the possible differentials in Q must have $\text{DP}_F = \varepsilon$. Denote by $D^\varepsilon \subset G^\ell \times G'$ the set of differentials over equal-length messages of length ℓ that have $\text{DP}_F = \varepsilon$ for some $\ell \in \mathbb{Z}_{\geq 1}$. Then, the set of pairwise differences of Q must be a subset of $D^\varepsilon \cup \{0\}$, i.e., $Q - Q \subseteq D^\varepsilon \cup \{0\}$. This leads us to the following theorem.

Theorem 2 (Maximum Size of Optimal Query Set). *Let F be an ε - Δ -universal KTH function. Let $Q \subseteq G^\ell \times G'$ be the set of queries to the bkh generating oracle. Let $D^\varepsilon \subset G^\ell \times G'$ be the set of equal-length differentials over messages of length ℓ that achieve $\text{DP}_F = \varepsilon$. The size of Q is upper bounded by that of D^ε , i.e.:*

$$\#Q \leq \#D^\varepsilon + 1.$$

Equality holds if and only if $D^\varepsilon \cup \{0\}$ is a subgroup of G^ℓ and Q a coset of that subgroup.

Proof. It is a standard result in additive combinatorics that for any finite subset Q of a group, $\#(Q - Q) \geq \#Q$, with equality holding if and only if Q is a coset of a finite subgroup (see, e.g., [TV06, Section 2, Exact inverse sum set theorem]). $\square$

Note that the above is only a necessary condition. The other necessary condition for an optimal success probability is that the solution sets of all pairs in Q are disjoint, i.e., $\#\mathcal{S}(Q) = \sum_{p \neq q \in Q} \#\mathcal{S}(\{p, q\})$.

3.3 Group Attacks

We leverage the inherent group structure of G to achieve a possible quadratic increase in the success probability with respect to the total number of message blocks used.

Definition 12. We define the function $\text{Pairs}_F : \mathcal{P}(\text{BS}(G, \kappa) \times G') \times [0, 1] \rightarrow \mathbb{Z}_{\geq 0}$ as the function that returns the number of pairs in the set $Q \subset \text{BS}(G, \kappa) \times G'$ that achieve a digest collision with probability greater than or equal to ρ with regards to the KTH function F :

$$\text{Pairs}_F(Q, \rho) = \#\{(p, q) \in Q^2 \mid \Pr[\text{Bad}_F(\{p, q\})] \geq \rho\}.$$

Theorem 3 (Group Attacks 1.0). *Let F be an ε - Δ -universal KTH function. Let $b \in G'$ be an output difference. Let $A_{b, \ell} = \{\mathbf{a} \in G^\ell \mid \text{DP}_F(\mathbf{a}, 0, b) = \varepsilon\}$ be the set of input differences of length ℓ that propagate to b through F with probability equal to ε . Define the set $Q = \langle A_{b, \ell} \rangle \times \{0\} \cup \langle A_{b, \ell} \rangle \times \{b\}$ as the set of queries sent to the bkh generating oracle. The following holds:*

$$\text{Pairs}_F(Q, \varepsilon) = \#A_{b, \ell} \# \langle A_{b, \ell} \rangle + \text{Pairs}(Q \setminus \langle A_{b, \ell} \rangle \times \{b\}, \varepsilon).$$

Proof. Since $\langle A_{b, \ell} \rangle$ is an abelian group, there are exactly $\#\langle A_{b, \ell} \rangle$ pairs $(a_1, a_2) \in \langle A_{b, \ell} \rangle^2$ with a fixed difference $a \in A_{b, \ell}$. Thus, there are $\#A_{b, \ell} \# \langle A_{b, \ell} \rangle$ pairs with a difference in $A_{b, \ell}$. The rest of the pairs have differential probability $\text{DP}_F(\mathbf{a}, 0, 0)$. Those who achieve a collision probability equal to ε are counted in $\text{Pairs}(Q \setminus \langle A_{b, \ell} \rangle \times \{b\}, \varepsilon)$. $\square$

We conjecture that it is very unlikely that $\text{Pairs}(Q \setminus \langle A_{b, \ell} \rangle \times \{b\}, \varepsilon) > 0$ unless F is specifically designed for it. We explain the intuition behind our conjecture as follows. There are currently two proposed KTH constructions: the serial and the parallel one. When we instantiate the serial and parallel with a public permutation, the best attacks are the ones that make use of single-block messages (i.e., elements of G). In order for an attack to succeed in that scenario, the attacker relies on a call to f to propagate to the target output difference b . As f is a bijection, the output difference for any input difference $a \neq 0$ cannot be 0. Therefore, all the pairs in $\langle A_{b, \ell} \rangle \times \{0\}$ have $\text{DP}_f = 0$. In the case that f is a public function and is used in the parallel construction, it is possible that $\text{DP}_f(a, 0) > 0$. However, for $\text{Pairs}(Q \setminus \langle A_{b, \ell} \rangle \times \{b\}, \varepsilon)$ to have a non-negligible number of pairs, f would need to have a class of input differences that achieve the best DP_f for both 0 and b , which seems like an undesirable property.

Corollary 4. *Group attacks 1.0 have $2\#\langle A_{b,\ell} \rangle$ queries if $b \neq 0$ and success probability*

$$\Pr[\text{Bad}_F^{bkh}(Q)] \approx \varepsilon \text{Pairs}_F(Q, \varepsilon) \approx \varepsilon \#A_{b,\ell} \# \langle A_{b,\ell} \rangle.$$

It is possible to combine the method used in catch attacks to increase the number of good pairs in $\text{Pairs}(Q, \varepsilon)$.

Theorem 4 (Group Attacks 2.0). *Let F be an ε - Δ -universal KTH function. Let $b \in G'$ be an output difference. Let $A_{b,\ell} = \{\mathbf{a} \in G^\ell \mid \text{DP}_F(\mathbf{a}, 0, b) = \varepsilon\}$ be the set of input differences of length ℓ that propagate to b through F with probability equal to ε . Let $B = \{b \in G' \mid \exists \mathbf{a} \in A_{b,\ell} \text{DP}_F(\mathbf{a}, 0, b) = \varepsilon\}$. Let $H_1, H_2 \leq G'$ be two subgroups of G' such that $H_1 + H_2 = \langle B \rangle$. Define the set $Q = \langle A_{b,\ell} \rangle \times H_1 \cup \langle A_{b,\ell} \rangle \times H_2$ as the set of queries sent to the bkh generating oracle. The following holds:*

$$\text{Pairs}_F(Q, \varepsilon) = \#A_{b,\ell} \# \langle A_{b,\ell} \rangle \# B + \text{Pairs}(\langle A_{b,\ell} \rangle \times H_1, \varepsilon) + \text{Pairs}(\langle A_{b,\ell} \rangle \times H_2, \varepsilon).$$

Proof. This is direct from the combination of Theorem 1 and 3. $\square$

Corollary 5. *Group attacks 2.0 have $\# \langle A_{b,\ell} \rangle (\#H_1 + \#H_2)$ queries and success probability*

$$\Pr[\text{Bad}_F^{bkh}(Q)] \approx \varepsilon \text{Pairs}_F(Q, \varepsilon) \approx \varepsilon \#A_{b,\ell} \# \langle A_{b,\ell} \rangle \# \langle B \rangle.$$

It is in the attacker's interest to cover all the values of B with the least amount of queries while increasing the amount of good pairs from $\langle A_{b,\ell} \rangle$.

Corollary 6. *An optimal group attack 2.0 uses a factor 2 more queries than the theoretical minimum.*

For a group attack 2.0 to be optimal, $A_{b,\ell} \leq G^\ell$ needs to be subgroup of G^ℓ and $B \cong H_1 \oplus H_2$ with $\#H_1 = \#H_2$. We give an example of applying these attacks in Section 6.

3.4 Translation Attacks

KTH functions have an offset-invariance property that may allow an attacker to achieve the following: Given a set of queries Q , it can build a set of queries Q' with size $\omega \#Q$ and success probability at least $\omega \Pr[\text{Bad}_F^{bkh}(Q)]$.

Lemma 1. *Let $\mathcal{S}$ be a solution set. Let $O \subset G^\kappa$ be a set of offsets. We denote by $\mathcal{D}$ the set of pairwise differences of the elements of $\mathcal{S}$, i.e., $\mathcal{D} = \mathcal{S} - \mathcal{S}$. The following holds:*

$$\forall o_1, o_2 \in O \quad \mathcal{S} + o_1 \cap \mathcal{S} + o_2 \neq \emptyset \iff o_2 - o_1 \in \mathcal{D}.$$

Proof. We start from the left-hand side. $\mathcal{S} + o_1 \cap \mathcal{S} + o_2 \neq \emptyset$ implies there exists $\mathbf{k}, \mathbf{k}^* \in \mathcal{S}$ such that $\mathbf{k} + o_1 = \mathbf{k}^* + o_2$. By rearranging the equation, we get $o_2 - o_1 = \mathbf{k} - \mathbf{k}^* \in \mathcal{D}$. We continue with the right-hand side. $o_2 - o_1 \in \mathcal{D}$ implies that $o_2 - o_1 = \mathbf{k} - \mathbf{k}^*$ for some $\mathbf{k}, \mathbf{k}^* \in \mathcal{S}$. By rearranging the equation, we get $\mathbf{k}^* + o_2 = \mathbf{k} + o_1$, implying that $\mathbf{k}^* + o_2 \in \mathcal{S} + o_1 \cap \mathcal{S} + o_2$. $\square$

Corollary 7. *Let $\mathcal{S}$ be a solution set and $\mathcal{D}$ its associated set of pairwise differences. Let O be a set of offsets for which $\mathcal{S} + o_1 \cap \mathcal{S} + o_2 = \emptyset$ for all unordered pairs $(o_1, o_2) \in O^2$ with $o_1 \neq o_2$. The following holds:*

$$o \in O \implies \forall o^* \in \mathcal{D} + o \quad o^* \notin O.$$

Theorem 5 (Translation Attack). *Let $Q = Q_1 \times Q_2 \subset \text{BS}(G, \kappa) \times G'$ be the set of queries sent to the bkh generating oracle. Let $\mathcal{S} \subset G^\kappa$ be the solution set of Q and $\mathcal{D} = \mathcal{S} - \mathcal{S}$ its associated pairwise differences set. Let $\mathbf{o} \in G^\kappa \setminus \mathcal{D}$ be an offset. We denote by $Q' = Q + (\mathbf{o}, 0)$ the set Q whose messages are translated by the value $\mathbf{o}$. The following holds.*

$$\Pr \left[\text{Bad}_F^{bkh} (Q \cup Q') \right] \geq 2 \Pr \left[\text{Bad}_F^{bkh} (Q) \right]$$

Proof. We apply Lemma 1 by taking $\mathbf{o}_1 = 0$ and $\mathbf{o}_2 = \mathbf{o}$. We get

$$\mathcal{S}(Q) \cap \mathcal{S}(Q') = \emptyset \implies \#\mathcal{S}(Q \cup Q') \geq 2\#\mathcal{S}(Q).$$

The pairs in Q and Q' contribute each $\#\mathcal{S}(Q)$ keys to the solution set. The inequality is due to the potential contributions of the pairs between Q and Q' . Since

$$\Pr \left[\text{Bad}_F^{bkh} (Q \cup Q') \right] = \frac{\#\mathcal{S}(Q \cup Q')}{\#G^\kappa},$$

we have proven the theorem. $\square$

The attacker can build a set of offsets $\mathcal{O}$ for which every translated query set $Q + (\mathbf{o}, 0)$ with $\mathbf{o} \in \mathcal{O}$ contributes $\#\mathcal{S}(Q)$ new keys, assuming we don't count the key contributions of the pairs between translated sets. We describe our method in Algorithm 1.

Algorithm 1: Generic Algorithm for Constructing an Offset Set for Translation Attacks

Inputs : The set of pairwise differences $\mathcal{D}$ of the solution set $\mathcal{S}$

Output : The set of offsets $\mathcal{O}$

```

Set  $\mathcal{O} = \{0\}$ 
Set  $\mathcal{C} = G^\kappa \setminus \mathcal{D}$ 
while  $\#\mathcal{C} > 0$  do
     $o \xleftarrow{\$} \mathcal{C}$ 
     $\mathcal{O} \leftarrow \mathcal{O} \cup \{o\}$ 
     $\mathcal{C} = \mathcal{C} \setminus (\mathcal{D} + o)$ 
end
return  $\mathcal{O}$ 

```

Lemma 2. *Let $\mathcal{S}$ be a solution set. Algorithm 1 returns an offset set $\mathcal{O}$ with the following bounds:*

$$\left\lceil \frac{\#G^\kappa}{\#\mathcal{S}^2} \right\rceil \leq \#\mathcal{O} \leq \left\lfloor \frac{\#G^\kappa}{\#\mathcal{S}} \right\rfloor.$$

Proof. We start with the lower bound. At each iteration of Algorithm 1, the size of $\mathcal{C}$ decreases by the quantity $\#\mathcal{C} \cap \mathcal{D} + o$. Since $\#\mathcal{D} + o \leq \#\mathcal{S}^2$, we can assume that each iteration removes the maximum possible offset candidates from $\mathcal{C}$. Now we prove the upper bound. By construction

$$\bigcup_{o \in \mathcal{O}} \mathcal{S} + o \subseteq G^\kappa,$$

$$\bigcap_{o \in \mathcal{O}} \mathcal{S} + o = \emptyset.$$

Thus, assuming $\bigcup_{o \in \mathcal{O}} \mathcal{S} + o = G^\kappa$ yields the upper bound. $\square$

As translation attacks make no assumption on $\mathcal{S}$, they can be combined with any other type of multi-query attacks in order to increase the success probability of the attacker.

3.5 Attacks on Concatenated KTH Functions

A common technique to create a stronger universal hash function is to concatenate the outputs of multiple calls to the same universal hash function with independent keys. We call this the augmentation of F . We prove how this method scales for KTH functions in a multi-query attack setting.

Definition 13 (Augmented KTH Function). Let $F: \text{BS}(\mathbb{G}, \kappa) \times \mathbb{G}^\kappa \rightarrow \mathbb{G}'$. Let $n \in \mathbb{Z}_{\geq 1}$ be the augmentation factor. We denote F augmented by n as $F^n: \text{BS}(\mathbb{G}, \kappa) \times \mathbb{G}^{\kappa n} \rightarrow (\mathbb{G}')^n$ and it is given by the following:

$$F^n_{\mathbf{k}}(\mathbf{m}) = (F(\mathbf{m} + \mathbf{k}_1), F(\mathbf{m} + \mathbf{k}_2), \dots, F(\mathbf{m} + \mathbf{k}_n)) .$$

Proposition 2. Let $n \in \mathbb{Z}_{\geq 1}$ be the augmentation factor. Let F^n be an augmented KTH function with F being ε - Δ -universal. Let $Q \subset \text{BS}(\mathbb{G}, \kappa) \times (\mathbb{G}')^n$ be the set of queries sent to the bkh generating oracle. The following holds:

$$\Pr[\text{Bad}_{F^n}^{\text{bkh}}(Q)] \leq \varepsilon^{n-1} \sum_{q \in (Q^*)^2} \Pr[\text{Bad}_F^{\text{bkh}}(q)] ,$$

where Q^* is the set that maximizes the following metric:

$$Q^* = \arg \max_P \sum_{p \in P^2} \Pr[\text{Bad}_F^{\text{bkh}}(p)] \quad \text{s.t.} \quad \#P \leq \#Q .$$

Proof. A collision in F^n requires that the same pair q collides for all n independent instances of F :

$$\Pr[\text{Bad}_{F^n}^{\text{bkh}}(Q)] \leq \sum_{q \in Q^2} \Pr[\text{Bad}_{F^n}^{\text{bkh}}(q)] .$$

Define the projection $\pi_i: \text{BS}(\mathbb{G}', \kappa) \times (\mathbb{G}')^n \rightarrow \text{BS}(\mathbb{G}', \kappa) \times \mathbb{G}'$ as $\pi_i((\mathbf{m}, \Delta)) = (\mathbf{m}, \Delta_i)$. We get the following derivation:

$$\begin{aligned} \sum_{q \in Q^2} \Pr[\text{Bad}_{F^n}(q)] &= \sum_{q \in Q^2} \prod_{i=1}^n \Pr[\text{Bad}_F(\pi_i(q))] \\ &\leq \varepsilon^{n-1} \sum_{q \in Q^2} \Pr[\text{Bad}_F(\pi_i(q))] , \end{aligned}$$

for some arbitrary value $1 \leq i \leq n$. By construction we have

$$\sum_{q \in Q^2} \Pr[\text{Bad}_F(\pi_i(q))] \leq \sum_{q \in (Q^*)^2} \Pr[\text{Bad}_F(q)] .$$

□

Q^* is the query set that produces the highest amount of collisions in F for a budget of at most $\#Q^*$ queries.

4 Generic Attacks on Parallel[f]

In this section, we prove the success probability of multi-query attacks that are generic to the parallelization of a public function.

4.1 Attacks Based on Unequal-Length Messages

We present the family of attacks that use unequal-length messages.

Proposition 3 (Unequal-Length Messages Attacks). *An attacker building a set of queries of ℓ queries, using only unequal-length messages, i.e.,*

$$Q = \cup_{i=1}^{\ell} \{0^i\} \times \{\Delta_i\},$$

has a collision probability over $\text{Parallel}[f]$ in the bkh setting that is upper bounded by:

$$\Pr \left[\text{Bad}_{\text{Parallel}[f]}^{bkh}(Q) \right] \leq (\ell - 1) \text{MIP}_f + \left(\frac{\ell^2 - 3\ell}{2} \right) \text{MNIP}_f.$$

Proof. We model the problem as follows.

The output of each $f(\mathbf{k}_i)$ can be seen as a discrete random variable with probability mass function (PMF) given by IP_f . We denote such variables as X_i for $1 \leq i \leq \ell$. The probability that the query $((0^i), \Delta_i)$ collides with $((0^j), \Delta_j)$ for $i < j$ is given by

$$\Pr \left[\sum_i^j X_i = \Delta_j - \Delta_i \right] = \text{IP} * \dots * \text{IP}(\Delta_j - \Delta_i),$$

where IP is convolved with itself $j - i$ times.

There are $\binom{\ell}{2}$ possible pairs of (i, j) . $\ell - 1$ of them have a difference in length $\lambda = 1$ and therefore collide with probability

$$\Pr \left[\sum_i^{i+1} X_i = \Delta_j - \Delta_i \right] = \text{IP}(\Delta_j - \Delta_i) \leq \text{MIP}_f.$$

There are $\frac{\ell^2 - 3\ell}{2}$ pairs with length difference $\lambda \geq 2$ and they all collide with probability at most

$$\Pr \left[\sum_i^j X_i = \Delta_j - \Delta_i \right] = \text{IP} * \dots * \text{IP}(\Delta_j - \Delta_i) \leq \text{MNIP}_f,$$

with $j \neq i + 1$. By applying Boole's inequality, we prove the bound. $\square$

The success probability of such attacks only depends on the number of queries and the choices of Δ offsets. We conjecture that the contribution of pairs of messages with length difference $\lambda \geq 2$ diminishes quickly. This is due to the fact that the more we convolve IP with itself, the more uniform the resulting PMF becomes.

Corollary 8. *Let $\Delta = \arg \max_x \text{IP}(x)$. Let $Q = \cup_{i=1}^{\ell} \{0^i\} \times \{i\Delta\}$, for some $\ell \in \mathbb{Z}_{\geq 1}$, and $i\Delta$ is the repeated addition of Δ with itself i times. The following holds:*

$$\Pr \left[\text{Bad}_{\text{Parallel}[f]}^{bkh}(Q) \right] \geq 1 - (1 - \text{MIP}_f)^{\ell-1}.$$

Proof. Each of the $\ell - 1$ pairs with length difference $\lambda = 1$ have a collision probability

$$\text{IP}((i+1)\Delta - i\Delta) = \text{IP}(\Delta) = \text{MIP}_f.$$

As the output difference with $\lambda = 1$ depends only on the key k_{i+1} , all $\ell - 1$ pairs are mutually independent. $\square$

The disadvantage of this type of attacks is that they increase quadratically in the number of messages blocks processed but only guarantee a linear increase in success probability.

4.2 Attacks Based on Two-Block Differences

We present a family of attacks that require only the knowledge of at least one differential with two active input blocks and high DP_f .

Proposition 4 (Two-Block Input Difference Attacks). *Let $\ell \in \mathbb{Z}_{\geq 1}$ be the attacker's chosen message length. Let $a \in G$ be a single-block input difference. The attacker constructs the set of queries $Q \subset G^\ell \times G'$ as follows:*

$$Q = \bigcup_{i=1}^{\ell} \{(0^{i-1}, a, 0^{\ell-i})\} \times \{\Delta_i\},$$

Then, the collision probability of Q through $\text{Parallel}[f]$ under the bkh model is upper bounded by:

$$\Pr \left[\text{Bad}_{\text{Parallel}[f]}^{bkh}(Q) \right] \leq \binom{\#Q}{2} \text{MNDP}_f.$$

Proof. For every pair in Q , whose messages are $(0^{i-1}, a, 0^{\ell-i})$ and $(0^{j-1}, a, 0^{\ell-j})$ with $i < j$, a collision occurs when the following holds:

$$f(a + k_i) - f(0 + k_i) + f(0 + k_j) - f(a + k_j) = \Delta_j - \Delta_i.$$

From Theorem 3 in [FRD23], each of these pairs has a collision probability of at most MNDP_f . By applying Boole's inequality we prove the upper bound. $\square$

Corollary 9. *It is possible to guarantee a linear lower bound in the number of queries. Let $(a, b) = \arg \max_{a^*, b^* \in G} \text{DP}(a^*, b^*)$. The query set Q is given by:*

$$Q = \bigcup_{i=1}^{\ell} \{(0^{i-1}, a, 0^{\ell-i})\} \times \{-b\}.$$

Finally, the attacker adds $\{0^\ell\} \times \{0\}$ to Q . Then, the following lower bound holds:

$$\Pr \left[\text{Bad}_{\text{Parallel}[f]}^{bkh}(Q) \right] \geq 1 - (1 - \text{MDP}_f)^{\ell-1}.$$

Proof. By construction, each pair of queries with indices $(0, i)$ has a collision probability $\text{DP}(a, b) = \text{MDP}_f$. Each of these pairs depend only on the values of the key k_i . As there are $\ell - 1$ such pairs and they are mutually independent, we prove the lower bound. $\square$

4.3 Mixed Strategies

One might wonder if mixed strategies should be considered. By mixed strategies, we mean that an attacker sends a query set Q for which the possible pairs in the set have a mixture of equal-length and unequal-length messages. We show that such a method is suboptimal.

Proposition 5. *The collision probability in the digest of $\text{Parallel}[f]$ in the bkh setting for an attacker using query set Q is upper bounded by the following:*

$$\Pr \left[\text{Bad}_{\text{Parallel}[f]}^{bkh}(Q) \right] \leq \max \left\{ \binom{\#Q}{2} \text{MDP}_f, (\#Q - 1) \text{MIP}_f + \left(\frac{\#Q^2 - 3\#Q}{2} \right) \text{MNIP}_f \right\}.$$

Proof. Let ℓ be the length of the longest message in Q . We denote by n_i the number of equal-length pairs of length i for $1 \leq i \leq \ell$ in Q . We denote by e the number of unequal-length pairs in Q . Since $\sum_i n_i = \binom{\#Q}{2} - e$, the following holds:

$$\Pr \left[\text{Bad}_{\text{Parallel}[f]}^{\text{bkh}}(Q) \right] \leq \left(\binom{\#Q}{2} - e \right) \text{MDP}_f + (e - 1) \text{MIP}_f + \left(\frac{e^2 - 3e}{2} \right) \text{MNIP}_f.$$

We denote this upper bound as a function of e , $g(e) = \left(\binom{\#Q}{2} - e \right) \text{MDP}_f + (e - 1) \text{MIP}_f + \left(\frac{e^2 - 3e}{2} \right) \text{MNIP}_f$ and we are interested in finding the maximum of g on the interval $e \in [0, \#Q]$. Taking the second derivative of g gives us:

$$g'' = \text{MNIP}_f > 0.$$

As g is convex, we prove the upper bound. □

5 Case Study: NH

While [FRD23] focused on Xoodoo as the primary instantiation, our attack framework applies to any KTH function. We first demonstrate it on $\text{NH}[w]$, the universal hash function used in UMAC [BHK⁺99], which provides a clean setting where the differential structure is fully understood and the attacks can be validated exactly.

$\text{NH}[w]$ is the universal hash function published by Black, Halevi, Krawczyk, Krovetz and Rogaway, and is used in UMAC [BHK⁺99]. We define a simplified variant, that omits the padding and treatment of unequal-length messages, using the KTH terminology. Let $f[w]: \mathbb{Z}/2^w\mathbb{Z} \times \mathbb{Z}/2^w\mathbb{Z} \rightarrow \mathbb{Z}/2^{2w}\mathbb{Z}$ with $f(x_1, x_2) = x_1 x_2 \bmod 2^{2w}$. Then, $\text{NH}[w]$ is given by:

$$\text{NH}[w] = \text{Parallel}[f[w]].$$

From [GAD23] we know that f has $\text{MIP}_f = 2^{-w+1}$ and it is given by the output value 0. Similarly, we know that it has $\text{MDP}_f = 2^{-w}$, with differentials of the form $((a, -a), 0), ((a, 0), 0), ((0, a), 0)$ and $((a, a), a^2)$ for all possible values of a achieving this value. We run experiments where we mount the attacks described in earlier sections. All of our experiments are done with either $w = 16$ or $w = 8$.

5.1 Experiment Using The Unequal-Length Attack

We build our message set as described in Corollary 8: Each message we send is of the form 0^i for $1 \leq i \leq \ell$. Since MIP_f is achieved using the output value 0, we are checking for direct collisions between the queries. Algorithm 2 describes at a high level the experiment performed using our Rust code.

We choose to fix the values of m_i and m_{i+1} to 0 as they do not have any effect on the experiment. This experiment is done for values $\ell \in \{2^1, 2^2, 2^3, \dots, 2^{10}\}$. In order to approximate the expected success probability, we run the experiment for $6 \cdot 10^6$ iterations and take the average.

Result of the unequal-Length Attack on $\text{NH}[16]$

We see that for the unequal-length attack, the only pairs that contributed to collisions in our experiments were pairs of the form $(0, 0), (0^i, 0)$. The resulting collision probability follows exactly the lower bound proven in Corollary 8.

Algorithm 2: Unequal-Length Experiment on $\text{NH}[w]$

Inputs : $\ell \in \mathbb{Z}_{\geq 2}$
Output : Found $\in \{\top, \perp\}$
Vec seen_values = {}
for $i \in (0 \dots \ell)$ **do**
 $k_i = \text{uniformValue}(0 \dots 2^w)$
 $k_{i+1} = \text{uniformValue}(0 \dots 2^w)$
 result = $\text{NH}[w](k_i, k_{i+1})$
 seen_values.push(result)
end
return repeat_found(seen_values)

5.2 Experiment on Two-Block Difference Attacks

We build our message set as described in Corollary 9: We choose $(1, -1)$ as our input difference and 0 as the output difference and perform the experiment as described in Algorithm 3.

5.2.1 Two-Block Input Differences Attack on $\text{NH}[16]$

The results of this attack show that it behaves similarly to the group attack and degrades from the upper bound for similar reasons. The reason for the small difference between this attack and the group attack is the fact that messages with two active differences have a collision probability of $\sum_b \text{DP}_f((1, -1), b)^2$ instead of $\text{MDP}_f = 2^{-16}$. From our experimental results, we can approximate the value of $\sum_b \text{DP}_f((1, -1), b)^2$ to be $2^{-16.64}$, which is very close to the upper bound of 2^{-16} .

Algorithm 3: Two-Block Input Difference Experiment on $\text{NH}[w]$

Inputs : $\ell \in \mathbb{Z}_{\geq 1}$
Output : Found $\in \{\top, \perp\}$
Vec seen_values = {}
Vec keys = {}
for $i \in (0 \dots 2\ell)$ **do**
 keys.push(uniformValue($0 \dots 2^w$))
end
u64 sum = 0
for $i \in (0 \dots \ell)$ **do**
 sum = sum + $\text{NH}[w](\text{keys}[2i], \text{keys}[2i + 1]) \bmod 2^{2w}$
end
seen_values.push(sum)
for $i \in (0 \dots \ell)$ **do**
 sum = sum - $\text{NH}[w](\text{keys}[2i], \text{keys}[2i + 1]) \bmod 2^{2w}$
 sum = sum + $\text{NH}[w](\text{keys}[2i] + 1 \bmod 2^w, \text{keys}[2i + 1] - 1 \bmod 2^w) \bmod 2^{2w}$
 seen_values.push(sum)
end
return repeat_found(seen_values)

In real attack scenarios the attacker has no access to the digest value `sum`. We make use of the knowledge of the digest value in order to decrease the number of computations. At each loop iteration we simply subtract the i th message's contribution to the digest and

add the contribution of the new translated message. This experiment is done for values $\ell \in \{2^1, 2^2, 2^3, \dots, 2^{10}\}$. In order to approximate the expected success probability, we run the experiment for $6 \cdot 10^6$ iterations and take the average.

5.3 Experiment on Group Attacks

We choose the input differences of the form $(a, -a)$ and output difference 0. The differences $(a, -a)$ form a subgroup of $(\mathbb{Z}/2^w\mathbb{Z})^2$ and we only care about output differences of value of 0, therefore we apply the group attack 1.0 as described in Theorem 3. Note that since $b = 0$, the query set we build is a coset.

Algorithm 4: Group Attack 1.0 Experiment on NH[w]

```

Inputs :  $\ell \in \mathbb{Z}_{\geq 2}$ 
Output: Found  $\in \{\top, \perp\}$ 

Vec  seen_values = {}
Vec  differences = uniformValuesNoRepeat( $\ell, 0 \dots 2^w$ )
 $k_1$  = uniformValue( $0 \dots 2^w$ )
 $k_2$  = uniformValue( $0 \dots 2^w$ )
for  $i \in (0 \dots \ell)$  do
    result = NH[ $w$ ]( $k_1 + \text{differences}[i] \bmod 2^w, k_2 - \text{differences}[i]$ 
                    $\bmod 2^w$ )
    seen_values.push(result)
end
return repeat_found(seen_values)

```

We also apply the group attack experiment to NH[8], NH[8]² and NH[8]³ to demonstrate the effects of Proposition 2. For $w = 16$, we run the experiments for the same values and iterations as the previous experiments. For $w = 8$, we run the experiments of $\ell \in \{2^1, 2^2, 2^3, \dots, 2^7\}$ and we do $3 \cdot 10^6$, $6 \cdot 10^6$ and 10^8 iterations respectively.

5.3.1 Results of the Group Attack on NH[16]

The group attack that we implement follows its upper bound, i.e., $\binom{\#Q}{2} 2^{-16}$ until $\#Q \approx 2^7$. The deviation from the upper bound can be explained by the following two properties. The first property is the fact that the input difference $(a, -a)$ propagates to an output difference 0, when $m_1 = m_2$, only when $a = k_2 - k_1$. Thus, it is independent of the absolute values of the message blocks. The second property is the fact that the closer we get to a success probability 1, the more likely it is that a new query reintroduces already existing input differences in Q .

5.3.2 Results of Group Attack 1.0 on Augmented NH[8]

As explained in Section 5.3.1, the value of a can be seen as the attacker's guess of the difference between the values of the keys k_1 and k_2 . From Proposition 2, we know that the same pair must collide for all independent instances of NH. Let $k_{i,j}$ represent the j th key of the i th NH instance. Then, this can only happen when $(k_{1,2} - k_{1,1}) = (k_{2,2} - k_{2,1})$ in the case of NH[8]² and $(k_{1,2} - k_{1,1}) = (k_{2,2} - k_{2,1}) = (k_{3,2} - k_{3,1})$ in the case of NH[8]³. This happens with probability 2^{-8} and 2^{-16} respectively and can be seen in our experimental results in Figure 6.

5.4 Results of the Experiments

We present in Figure 4 the results of all the experiments on NH[16] where the cost metric is the number of queries in Q , in Figure 5 the same results but using the *data complexity* of Q , i.e., the total number of blocks in Q , and in Figure 6 the results of the experiments on the augmentation of NH[8].

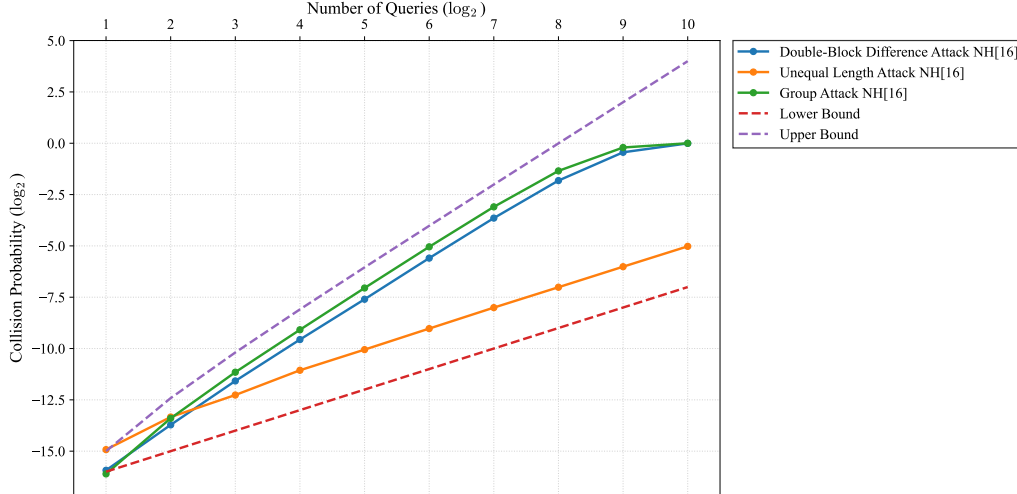


Figure 4: Result of the experiments on NH[16]. The lower bound is given by the curve $1 - (1 - \text{MDP}_f)^{\ell-1}$. The upper bound is given by the curve $\binom{\#Q}{2} \text{MIP}_f$.

6 Case Study: Xoodoo[3]

XOODOO is the 384-bit iterated permutation used in the doubly extendable cryptographic keyed function Xoofff [DHVV18b]. The specifications of XOODOO are attached in Appendix A. Its round function uses the $\chi: \mathbb{F}_2^3 \rightarrow \mathbb{F}_2^3$ function on three bits. It is of degree 2 and its parallel application to the 128 3-bit columns of the state is the round function's non-linear layer. This means that the set of compatible input differences with a given output difference forms an affine space (and vice versa) [Dae95, Section 6.9.1]. Daemen et al. provide a list of 3-round trail core classes in the Xoodoo GitHub repository [DHVV18a]. Among these trail cores, the authors identify two classes of trail cores that they name *Single-Orbital Fan* and *Vortex* [DHVV18b, Figure 8].

6.1 Vortex and Single-Orbital Fan Trail Cores

We will use the Vortex and Single-Orbital Fan trail cores to perform our attacks. Both of these types of trail cores have a restriction weight of 36. The paper of Bordes et al. investigated the clustering and round differential independence for 3-round trail cores up to weight 50. Their results show that for those trails, the probability of the differential defined by the trail is given by the probability of that trail (i.e., no clustering) and that it was given by $\text{DP} = 2^{-w}$, where w is the restriction weight of the trails in the trail core [BDKV21]. We perform our experiments under the assumption that $\text{MDP}_f = 2^{-36}$ and is given by the differential defined by the Vortex and Single-Orbital Fan trail cores. This means that we assume that there are no trail cores with weight above 50 that exhibit strong enough clustering to form a differential with $\text{DP} > 2^{-36}$.

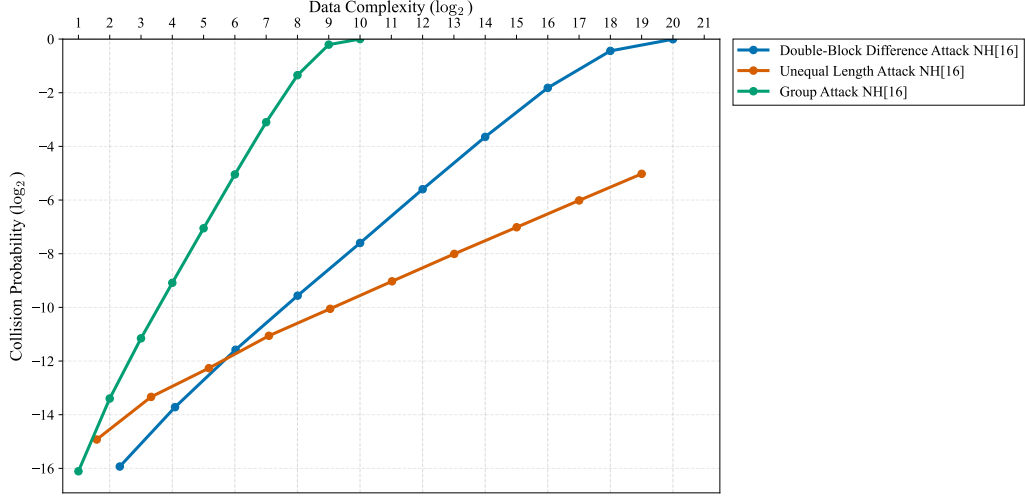


Figure 5: Result of the experiments on NH[16] using data complexity as a metric instead of number of queries. The data complexity is given by the number of NH[16] calls. For the double-block difference attack it is $\#Q^2 + 1$, for the group attack it is $\#Q$ and for the unequal-length attack it is $\frac{\#Q^2 + \#Q}{2}$.

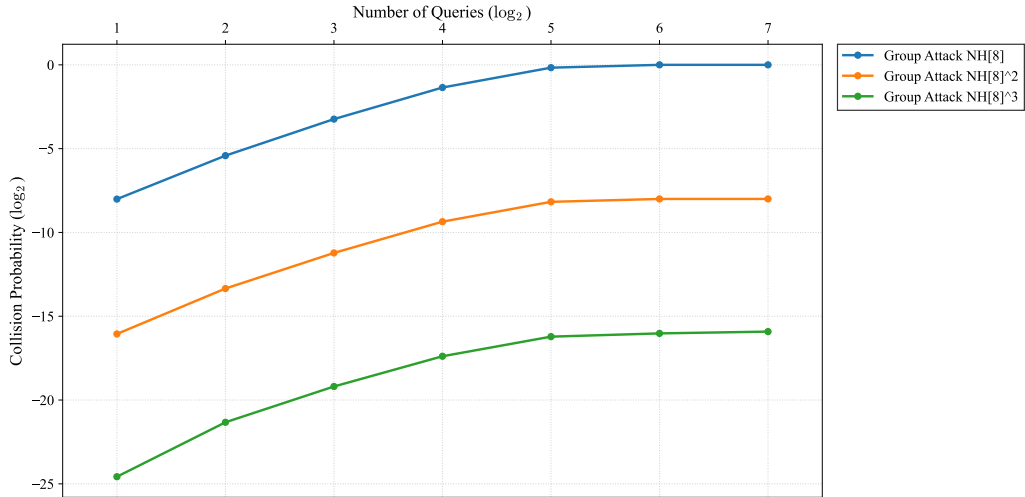


Figure 6: Showcase of the effect of Proposition 2 on the expected collision probability of Q in NH[8], NH[8]² and NH[8]³.

6.2 Building Query Sets From Trail Cores

We performed experiments on XOODOO[3], the 3 rounds version of XOODOO, by building group attacks 2.0 using the Single-Orbital fan and Vortex trail cores. In Xoofff, the permutation used is XOODOO[6], hence this should be seen as an experiment on the round-reduced variant of the compression phase of Xoofff.

Each active column at the output of the first non-linear layer contributes an affine space of size 2^2 to the space of compatible input differences with the trail core. The union of that affine space with the vector space that it is parallel to results in the complete vector space over 3 bits. More generally, for a given output/input difference, the affine space of compatible input/output differences is of dimension $2d$, where d is the number of active columns. The Vortex trail core has 6 active columns at the output difference b_1 in the differential over chi in the first round, therefore, the size of the set of compatible input differences with b_1 is $\#A_{b_1} = 2^{12}$. The set of compatible output difference at the output of the third round B has dimension 12. It is possible to split it into two subspaces H_1, H_2 of dimension 6. Similarly, for the Single-Orbital Fan, $\#A_{b_1} = 2^4$ and $\#B = 2^{28}$ and can be split into two subspaces of dimension 14. We note that $\langle A_{b_1} \rangle = A_{b_1} \cup (A_{b_1} \oplus a)$ for any $a \in A_{b_1}$. Therefore, we apply a small optimization and choose the query set to be

$$Q = A_{b_1} \times H_1 \cup (A_{b_1} \oplus a) \times H_2.$$

This is a small optimization over the query set defined in Theorem 4 which saves a factor 2 in the size of the query set but still preserves the same number of good pairs. So the query sets for the Vortex and Single-Orbital Fan trail cores both have size 2^{19} .

6.3 Group Attacks 2.0 on Xoodoo[3]

Unlike real attack scenarios, in our experiments we have the ability to check how the messages evolve through the rounds of the permutation. We take the following approach to simulate the attack, as explained in Algorithm 5.

Algorithm 5: Group Attack 2.0 Experiment on XOODOO[3]

```

Inputs :  $a_1, a_2 \in \{0, 1\}^{384}$ 
Output : found  $\in \{\top, \perp\}$ 

bool    found =  $\perp$ 
XoodooState key = uniformValue( $0 \dots 2^{384}$ )
Vec    states = compatibleSpace( $a_1$ )
states = add(key, states)
nonLinearLayer(states)
Vec    pair = add( $a_1$ , states)
permute(states)
permute(pair)
for  $i \in (0 \dots \text{states.size})$  do
    if  $\text{states}[i] \oplus \text{pair}[i] == a_2$  then
        found =  $\top$ 
    end
end
return found

```

We have verified that when we input Q we get 2^{12} pairs with the correct difference for the Vortex trail core and 2^4 pairs for the Single-Orbital Fan. Therefore, we artificially generate them at the output of the first non-linear layer to save us from searching for them. This is acceptable due to the fact that adding a key to the columns of the trail core

simply shuffles the pairs. Denote by $V + o$ the affine space that represents the compatible input differences with the output difference b_1 after the first round. The pairs with good differences are of the form $(v, v') \in (V + o) \times V$ for the bits in the columns that are targeting the trail cores. Addition with a key value $k \in (V + o)$ will transform the pairs to $(v + k, v' + k) \in V \times (V + o)$, and addition with a key value $k \in V$ will transform the pairs to $(v + k, v' + k) \in (V + o) \times V$. Since $k \in V \cup (V + o)$ and we use all pairs in $(V + o) \times V$, we can use the shortcut above.

6.4 Improved Group Attack on Xoodoo[3]

We noticed in our previous experiment that when we do get a collision using group attack 2.0, we get a very large amount of collisions at once and they are all mutually dependent. Hence, for some keys we get a lot of collisions at once and for most we get none. Therefore, we set up another experiment where we take the same approach for generating $\langle A_{b_1} \rangle$ as generating $\langle B \rangle$, by computing H'_1, H'_2 such that $H'_1 + H'_2 = \langle A_{b_1} \rangle$ and sending as a query set $Q = H'_1 \times H_1 \cup H'_2 \times H_2$. We run the same experiment as in Algorithm 5 but this time we do not apply the pairs shortcut like in the previous experiment.

Algorithm 6: Improved Group Attack 2.0 Experiment on XOODOO[3]

```

Inputs :  $a_1, a_2 \in \{0, 1\}^{384}$ 
Output : found  $\in \{\top, \perp\}$ 

bool    found =  $\perp$ 
XoodooState key = uniformValue( $0 \dots 2^{384}$ )
(Vec, Vec) (H'_1, H'_2) = compatibleSumSpace( $a_1$ )
(H'_1, H'_2) = (add(key, H'_1), add(key, H'_2))
nonLinearLayer(H'_1)
nonLinearLayer(H'_2)
permute(H'_1)
permute(H'_2)
for  $i \in (0 \dots H'_1.size)$  do
    for  $j \in (0 \dots H'_2.size)$  do
        if  $H'_1[i] \oplus H'_2[j] == a_2$  then
            found =  $\top$ 
        end
    end
end
return found

```

6.5 Result of the Experiments

We present in Figure 7 all the results of our experiments on XOODOO[3].

Group Attack 2.0 on Xoodoo[3]

Before we started the experiments, we expected the attack to succeed with probability close to 1 in both the Vortex and Single-Orbital fan trail cores. However, while running the experiment we saw that in the Vortex trail core, 2^{12} pairs survived the first round, and either 2^{12} , 2^{11} or 0 pairs survived the second round. Similarly, for the Single-Orbital fan trail core, 2^4 pairs survive the first round, and either 2^4 or 0 pairs survived the second round. This is due to the fact that χ is of degree 2 and the conditions on the key bits and message bits to propagate to the correct difference after a single round are linear.

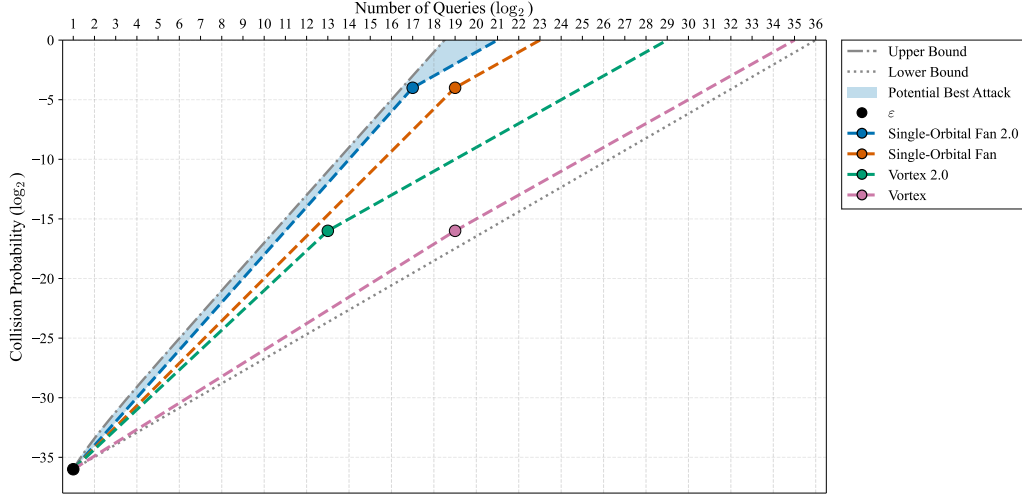


Figure 7: Experimental results on Xoodoo[3]. The upper bound is given by $\binom{\#Q}{2}2^{-36}$ while the lower bound is given by $\#Q2^{-36}$.

However, when we bring the conditions of propagation from the second round to the start of the first round, these conditions become quadratic. We expect them to be of the form $k_i + m_j k_l = 1$. As the bits of m_i have equal chance to be 0 or 1 in our query messages, this would explain why we see that all, half or none of the pairs propagate to the correct output difference after the second round. Our experiments also show that the success probability is approximately 2^{-16} for the Vortex trail core and 2^{-4} for the Single-Orbital Fan trail core.

Improved Group Attack 2.0 on Xoodoo[3]

We were interested in seeing if all the surviving pairs in the previous experiment were due to dependency of events. We confirm this conjecture by constructing a query set where only one pair survives the first round. Our results show that the expected probability of success is the same in both experiments, confirming our conjecture. Therefore, the large amount of good pairs at the input of the first round does not increase the success probability. Hence, we were able to improve the query cost by a quadratic factor relative to the size of the compatible input differences. The most efficient attack is based on the Single-Orbital Fan trail core. It requires 2^{19} queries and succeeds with probability 2^{-4} , resulting in an attack that is only a factor $\approx 2^4$ away from the possible best attack.

7 Conclusion and Discussion

7.1 Characterization of the Gap

Our experiments show that a quadratic increase in the number of queries is approximately possible, but a gap remains. We identify two structural sources of this gap. The first is that the set of differentials with probability ϵ does not always form a subgroup. From Theorem 2, the optimal query set requires the differentials to form a subgroup of the query space. When this condition is only approximately met, the number of good pairs is reduced. The second is overlap in solution sets. Even when the differentials form a perfect subgroup, the solution sets of different pairs may intersect, causing the same keys to be counted multiple times. This overlap manifests differently depending on the instantiation. For NH,

the differentials $(a, -a)$ form a subgroup of $(\mathbb{Z}/2^w\mathbb{Z})^2$, but as the query set grows and the success probability approaches 1, new queries increasingly reintroduce existing input differences, causing unavoidable solution set overlap. For XOODOO[3], the overlap has a deeper structural cause. The nonlinear layer χ has algebraic degree 2, so the conditions for correct propagation through a single round are linear in the key bits. Composing two rounds transforms these into quadratic expressions. Since the message bits in our query set take values 0 and 1 with equal probability, pairs succeed or fail together based on shared key-dependent conditions, rather than independently. This makes the solution set overlap inherent to the degree-2 structure, not an artifact of our query set construction.

7.2 Conclusion

We presented in this paper a series of multi-query attacks on KTH functions that vary in generality from applying to all KTH functions to attacks that apply only on the parallel construction. We argued that it is not obvious that an attacker can reach the theoretical quadratic bound for multi-query attacks. Still, we presented attacks that we thought could potentially reach a quadratic increase assuming that all the necessary conditions were met. By applying them to well-known KTH functions such as NH and Parallel[XOODOO[3]], we showed that our attacks actually achieve an increase very close to quadratic for queries that form differentials with probability ε . Perhaps the most surprising result is how effective the catching mechanism of the group attack 2.0 is for iterated permutations over $\{0, 1\}^{2n}$, if the round function and its inverse have algebraic degree 2, since it is always possible to split the output space into two subgroups of size 2^n such that their sum is equal to the original group. The potential quadratic increase coming from that is deterministic and therefore may introduce a new criterion for trail search when designing new permutations for KTH functions. Specifically, the dangerous part is the existence of a trail core where a large part of the active sboxes are in the last round, the more active boxes there are, the closer the attack gets to being optimal. However, just like many other types of cryptanalytical attacks against cryptographic permutations, we can see that increasing the number of rounds makes performing these attacks very difficult. As the number of rounds increases, it becomes more difficult to have disjoint solution sets and it becomes computationally infeasible to know how the differential trails look like.

A Xoodoo Specification

We quote the specification of the XOODOO round function, taken verbatim from the Xoodoo Cookbook [DHP⁺18].

XOODOO is a family of permutations parameterized by its number of rounds r and denoted XOODOO[r].

XOODOO has a classical iterated structure: It iteratively applies a round function to a state. The state consists of 3 equally sized horizontal *planes*, each one consisting of 4 parallel 32-bit *lanes*. Similarly, the state can be seen as a set of 128 *columns* of 3 bits, arranged in a 4×32 array. The planes are indexed by y , with plane $y = 0$ at the bottom and plane $y = 2$ at the top. Within a lane, we index bits with z . The lanes within a plane are indexed by x , so the position of a lane in the state is determined by the two coordinates (x, y) . The bits of the state are indexed by (x, y, z) and the columns by (x, z) . *Sheets* are the arrays of three lanes on top of each other and they are indexed by x . The XOODOO state is illustrated in Figure 8.

The permutation consists of the iteration of a round function R_i that has 5 steps: a mixing layer θ , a plane shifting ρ_{west} , the addition of round constants ι , a non-linear layer χ and another plane shifting ρ_{east} .

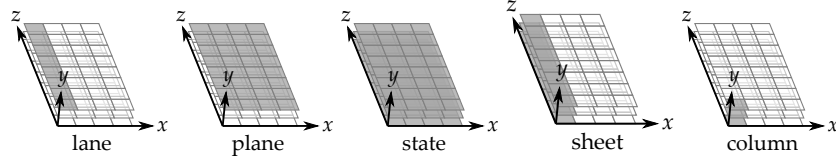


Figure 8: Toy version of the XOODOO state, with lanes reduced to 8 bits, and different parts of the state highlighted. [DHP⁺18]

We specify XOODOO in Algorithm 7, completely in terms of operations on planes and use thereby the notational conventions we specify in Table 2.

References

- [BCK96] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Pseudorandom functions revisited: The cascade construction and its concrete security. In *37th Annual Symposium on Foundations of Computer Science, FOCS '96, Burlington, Vermont, USA, 14-16 October, 1996*, pages 514–523. IEEE Computer Society, 1996.
- [BDH⁺17] Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Farfalle: parallel permutation-based cryptography. *IACR Trans. Symmetric Cryptol.*, 2017(4):1–38, 2017.
- [BDKV21] Nicolas Bordes, Joan Daemen, Daniël Kuijsters, and Gilles Van Assche. Thinking Outside the Superbox. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 337–367. Springer, 2021.
- [BHK⁺99] John Black, Shai Halevi, Hugo Krawczyk, Ted Krovetz, and Phillip Rogaway. UMAC: Fast and Secure Message Authentication. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 216–233. Springer, 1999.
- [Dae95] Joan Daemen. *Cipher and hash function design, strategies based on linear and differential cryptanalysis*, PhD Thesis. K.U.Leuven, 1995. <http://jda.noekeon.org/>.
- [DHP⁺18] Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Xoodoo cookbook. Cryptology ePrint Archive, Paper 2018/767, 2018. <https://eprint.iacr.org/2018/767>.
- [DHP⁺20] Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Xoodyak, a lightweight cryptographic scheme. *IACR Trans. Symmetric Cryptol.*, 2020(S1):60–87, 2020.
- [DHVV18a] Joan Daemen, Seth Hoffert, Gilles Van Assche, and Ronny Van Keer. DC-Xoodoo-3r.txt. <https://github.com/KeccakTeam/Xoodoo/blob/master/XooTools/Trails/DC-Xoodoo-3r.txt/>, 2018.

Table 2: Notational conventions [DHP⁺18]

A_y	Plane y of state A
$A_y \lll (t, v)$	Cyclic shift of A_y moving bit in (x, z) to position $(x + t, z + v)$
$\overline{A_y}$	Bitwise complement of plane A_y
$A_y + A_{y'}$	Bitwise sum (XOR) of planes A_y and $A_{y'}$
$A_y \cdot A_{y'}$	Bitwise product (AND) of planes A_y and $A_{y'}$

Algorithm 7: Definition of XODOO[r] with r the number of rounds [DHP⁺18]

Parameters: Number of rounds r
for Round index i from $1 - r$ to 0 **do**
 $A = R_i(A)$
end for

Here R_i is specified by the following sequence of steps:

θ :
 $P \leftarrow A_0 + A_1 + A_2$
 $E \leftarrow P \lll (1, 5) + P \lll (1, 14)$
 $A_y \leftarrow A_y + E$ for $y \in \{0, 1, 2\}$
 ρ_{west} :
 $A_1 \leftarrow A_1 \lll (1, 0)$
 $A_2 \leftarrow A_2 \lll (0, 11)$
 ι :
 $A_0 \leftarrow A_0 + C_i$
 χ :
 $B_0 \leftarrow \overline{A_1} \cdot A_2$
 $B_1 \leftarrow \overline{A_2} \cdot A_0$
 $B_2 \leftarrow \overline{A_0} \cdot A_1$
 $A_y \leftarrow A_y + B_y$ for $y \in \{0, 1, 2\}$
 ρ_{east} :
 $A_1 \leftarrow A_1 \lll (0, 1)$
 $A_2 \leftarrow A_2 \lll (2, 8)$

Table 3: The round constants c_i with $-11 \leq i \leq 0$, in hexadecimal notation (the least significant bit is at $z = 0$) [DHP⁺18].

i	c_i	i	c_i	i	c_i	i	c_i
-11	0x00000058	-8	0x000000D0	-5	0x00000060	-2	0x000000F0
-10	0x00000038	-7	0x00000120	-4	0x0000002C	-1	0x000001A0
-9	0x0000003C0	-6	0x00000014	-3	0x000000380	0	0x00000012

- [DHVV18b] Joan Daemen, Seth Hoeffert, Gilles Van Assche, and Ronny Van Keer. The design of Xoodoo and Xooff. *IACR Trans. Symmetric Cryptol.*, 2018(4):1–38, 2018.
- [DR05] Joan Daemen and Vincent Rijmen. The Pelican MAC Function. *IACR Cryptol. ePrint Arch.*, 2005:88, 2005.
- [DV12] Joan Daemen and Gilles Van Assche. Differential Propagation Analysis of Keccak. In Anne Canteaut, editor, *Fast Software Encryption - 19th International Workshop, FSE 2012, Washington, DC, USA, March 19-21, 2012. Revised Selected Papers*, volume 7549 of *Lecture Notes in Computer Science*, pages 422–441. Springer, 2012.
- [FRD23] Jonathan Fuchs, Yann Rotella, and Joan Daemen. On the security of keyed hashing based on public permutations. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 607–627. Springer, 2023.
- [GAD23] Koustabh Ghosh, Parisa Amiri-Eliasi, and Joan Daemen. Multimixer-128: Universal keyed hashing based on integer multiplication. *IACR Trans. Symmetric Cryptol.*, 2023(3):1–24, 2023.
- [GDM19] Aldo Gunsing, Joan Daemen, and Bart Mennink. Deck-Based Wide Block Cipher Modes and an Exposition of the Blinded Keyed Hashing Model. *IACR Trans. Symmetric Cryptol.*, 2019(4):1–22, 2019.
- [GFAD23] Koustabh Ghosh, Jonathan Fuchs, Parisa Amiri-Eliasi, and Joan Daemen. Universal hashing based on field multiplication and (near-)mds matrices. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings*, volume 14064 of *Lecture Notes in Computer Science*, pages 129–150. Springer, 2023.
- [Sti95] Douglas R. Stinson. On the Connections Between Universal Hashing, Combinatorial Designs and Error-Correcting Codes. *Electron. Colloquium Comput. Complex.*, 2(52), 1995.
- [TV06] Terence Tao and Van H. Vu. *Additive Combinatorics*, volume 105. Cambridge University Press, 2006.